

BendRT: A Parallel Runtime for CPUs and GPUs

Victor Taelin
Higher Order Company
Rio de Janeiro, Brazil
taelin@higherorderco.com

AI DISCLOSURE. Bend and its runtime were designed by the human author. This paper was written with the assistance of Claude (Fable 5, Anthropic), based on the author’s code and design notes, and thoroughly reviewed by the author.

Abstract

Bend is a pure functional programming language with an affine dependent type system. This paper describes BendRT, its runtime. The compiler emits one standalone C file per program, and that file runs the same code sequentially, in parallel on multicore CPUs, and on Apple-Silicon GPUs through Metal, with no garbage collector and no user-written kernels. The programmer writes two annotations: the fork `let`, which promises that its calls run in parallel and split the work into roughly equal halves, and `f!(x)`, which requests that a call run on the GPU. Tasks fork along fork `lets` until the lanes are saturated, then every lane runs its tasks to completion, sequentially. The scheduler contains no work stealing and no shared task queues; this is what lets the same phase bodies run as CPU worker threads and as GPU dispatches, at the price that a program with unequal forks loses parallel speed. Affinity makes every value uniquely owned by default, so matching frees, deallocation is compiled code, and a forked task carries no synchronization; where the types license reuse, the compiler places a counted share or a borrow, never a tracing collector. We describe the compiled form, the memory layout, the task system, the sequential machine, and the Metal backend, and report benchmarks on an Apple M4 Mac Mini.

1 Introduction

Bend is a pure functional language: programs are recursive functions over algebraic datatypes, and its type system, an affine dependent type theory developed in a companion paper [14], makes every live value consumed at most once unless its type is of the reusable kind `Data` where reuse is licensed, the compiler places a borrow (a read in place) or a counted share, each decided at compile time. A functional program is an evaluation strategy away from a parallel one, since independent subexpressions may run in any order on any core; historically the speedups have foundered on garbage collection, which couples every core to a shared collector, on task scheduling, which either leaves cores idle or pays for work stealing, and on GPUs, which want flat memory, bounded recursion, and kernels. BendRT is built from three parts that fit together.

Programs are worklist segments. The compiler rewrites every function until each call sits in one of three runnable shapes, a tail call, a cut (one call whose continuation is

named), or one parallel fork, and everything else is call-free straight-line code (Section 3). Each function then compiles to one segment of a single worklist machine that has no C call stack: calls are jumps, and every call site carries both a sequential and a parallel reading of the same text (Section 8). This one shape serves every executor: it is the step the scheduler advances one fork at a time, the body a lane runs to completion, and, having no native recursion and no captured environments, the code a GPU compiler accepts.

The scheduler is contention-free. Evaluation is bulk synchronous [17]: tasks fork exponentially along fork `lets` until the lanes are saturated, then every lane runs its tasks to completion (Section 7). There is no work stealing, no shared deque, no balancing of any kind: tasks live in a fixed grid of rings, one per lane, and the only global motion is an $O(1)$ transposition of the grid’s index map. Nothing contends and each phase is a closed step, which is what lets the same scheduler run as persistent CPU threads and as GPU dispatches. The price is that the runtime never rescues a program whose forks are unequal (Section 2.2).

Affinity supplies the memory story. A match consumes its scrutinee, so the runtime frees nodes exactly where ownership says they die, and dropping a value is one iterative walk that threads its worklist through the dying nodes themselves. A forked task owns its arguments outright, so forking synchronizes nothing: the only cross-thread protocol is the delivery of results into join tasks. Where the compiler’s own analysis finds sharing, it places a one-word counted redirect beside the shared term, and where a callee provably only reads, it lends the value instead; there is no tracing, no epochs, and no count on any unshared value.

Five words recur from here on, so we fix them now. A *task* is a forked call waiting to run; the *frontier* is the set of live tasks. Evaluation alternates three phases: *seed* runs the small frontier on one lane group, *grow* runs tasks one fork step at a time to widen it, and *work* runs each task to completion, sequentially, one lane per task. Section 7 defines all five precisely; until then these one-line meanings suffice.

The paper walks the machine bottom-up: the source-level contract (Section 2), the compiled form (Section 3), terms and memory (Section 4), run-time ownership (Section 5), arrays (Section 6), the task system (Section 7), sequential execution (Section 8), the Metal backend (Section 9), the event loop (Section 10), benchmarks (Section 11), related work (Section 12), and limitations (Section 13). Readers of

the author’s earlier work may expect interaction nets [8, 13] in this stack; there are none (Section 12).

2 The Contract with the Programmer

2.1 Affinity, from the Types

Bend’s checker tracks every use. A live value is consumed at most once, with one way around it: a binder marked `+` may be consumed freely, and its type must be of the reusable kind `Data`, a *kind* every datatype declares and the checker verifies at each constructor, so that no closure ever sits inside a `Data` value (the companion paper develops the discipline). Kinds are erased at runtime; a `+` variable is simply emitted at every use. The checker is authoritative about cost: a compiler may drop a cost the source spelled, never add a clone or a retain the source did not. The runtime exploits that latitude. The default is a move: passing a value transfers ownership, and no deep cloner exists anywhere in the runtime. Where a value genuinely gains a second owner, the compiler emits a *keep*: a one-word counted redirect minted beside the term, bumped and decremented only at real sharing sites. And wherever an extra use provably only reads, the compiler passes a *borrow* instead: the callee walks the owner’s tree in place through bare loads and the owner frees it once, after the last read. Three runtime consequences organize everything that follows:

1. An unshared value has exactly one owner, so the runtime may mutate and free it without synchronization, and pattern matching compiles to load-then-free.
2. Every share and borrow site is static, placed by the compiler’s own analyses from the types and the use counts, so a program’s memory traffic is readable off its source; a program that never reuses compiles to pure move-shaped code, with no count anywhere.
3. Which constructor types can be shared at all is decided once, for the whole program: only those *wear* reference counts, and everything else is built and consumed through plain stores and loads (Section 5).

2.2 The Fork

The fork `let` spells its members in one binding,

```
a b = sum(l) sum(r)
a + b
```

and means: these calls run in parallel, and they split the work into roughly equal halves. That promise is the *equal-halves contract*, and the whole task grid is built on trusting it (Section 7): sibling computations are assumed to split at the same arity all the way down, so a saturated grid holds equal hands. The parser builds one `Let` binding `n` names to `n` values; the compiler turns it into one fork whose members become tasks and whose continuation becomes their join. Nothing else in the language creates parallelism, and the runtime trusts the promise absolutely (Section 1): balance is the program’s job, never the scheduler’s. When the assumption fails, lanes idle at the end of a work turn,

and the runtime must not correct that: any such correction is treated as a compiler bug, not a scheduling feature. The honest idioms are teachable, and they cover practice so far: fork equal halves of the data or index space; sequence full-width phases instead of forking phases against each other; keep light work out of forks.

2.3 The Placement Mark

The second and last annotation is the call mark `f!(x)`: run this call on the GPU. The mark has no type rule or evaluation rule of the language that can observe it; it travels as one bit on the reference itself. Its meaning is scoped: the runtime never computes on two devices at once, so a marked call is honored when the event loop meets it at a sequential program point, where nothing else is running (Section 10); there it detaches the whole call to the Metal evaluator, which computes the answer on the shared heap and hands it back. Inside a running computation the mark degrades gracefully: a marked call in the parallel world spawns its task rather than nesting into it, and in the sequential world it is inert. On a build or machine without a usable GPU the mark is inert everywhere. There is no automatic backend pick: execution is the parallel CPU driver unless the programmer asked otherwise (`-gpu on`, default when a device probes; asking for a device that is not there is a refusal, not a fallback). Section 9.5 describes the hand-off.

3 The Shape of a Compiled Program

3.1 One File, Three Executors

The compiler emits one standalone C file: the runtime (a single fixed template, spliced in verbatim) followed by the program’s constructor and function tables, its compiled segments, and the effect handlers it imports. The same file compiles as plain C for the CPU build and as Objective-C for the Metal host; a GPU-capable binary then compiles *its own source file* as the Metal shader library at launch, so the device kernels are, by construction, the same code the host runs. Every build is one compiler invocation and every binary is self-contained: it ships no separate kernel. The three executors, sequential CPU, parallel CPU, and Metal GPU, are selected at run time (`-parallel`, `-threads`, `-gpu`): every call site is emitted in both a sequential and a parallel reading behind one runtime flag (Section 8), so one text serves all three. A second, sequential backend emits the same compiled book as plain JavaScript over the host’s garbage collector; it shares every compiler pass and is out of scope here.

3.2 Carbo: Tail, Cut, Fork

Before emission, every function reachable from `main` is rewritten into the form the runtime schedules, called *Carbo*: an outer tree of lambdas and matches, then straight-line lets ending in one of three shapes. A *tail* is a saturated call in return position. A *cut* is one call whose result is named and whose continuation is a minted sequential definition.

A *fork* is a chain of two or more calls whose results feed a named joiner. Everything else, constructor builds, arithmetic, erased proofs, is call-free expression code. Lambdas lift into top-level definitions over their captures, so a closure value is an under-saturated spine of an ordinary definition, applied through the same call protocol as everything else; dynamic applications route through a synthesized apply step. The rewrite is one recursive pass per definition, minting continuations, joiners, lifted matches and closure bodies as it goes; an inliner then splices small, fork-free case trees into their callers, and unrolls closed recursion over constants into straight-line code, so the format’s cuts are only the calls that genuinely must suspend.

Each definition in this form compiles to exactly one *segment* of the worklist machine (Section 8). Which world runs a segment is the scheduler’s choice, never the code’s: the grow phase of Section 7 runs fork steps to widen the frontier, and the work phase runs the same segments sequentially to drain it. A definition that can never fork is marked once in a table, closed under references, and the scheduler skips it during growth: it can never widen the frontier, so it waits for the work phase and runs whole.

3.3 Example

The canonical tree sum:

```
type Tree:
  Leaf{x: U32}
  Node{l: Tree, r: Tree}

assert sum:
  forall t: Tree
    U32

def sum(t):
  match t:
    case Leaf{x}:
      x
    case Node{l, r}:
      &a: U32 = sum(l)
      &b: U32 = sum(r)
      a + b
```

The declaration is an assert (the type) plus a def (the fill); the fork `let` promises two equal halves. Carbonization leaves the case tree in place, turns the fork into two calls joined by a minted definition `sum$j0(a, b) = a + b`, and the emitter prints one segment of this shape (simplified, names shortened; `seq` is the machine’s world flag):

Kind	Aux + location	Node slots
word	32-bit value or small Nat, unboxed	none
constructor	ctor id + loc	fields
packed ctor	ctor id; payload in loc	none
closure	fn id + loc	captured args
task	fn id + loc	args, cont, idx rem
array	cell class + loc	one term per word
buffer	cell class + loc	u32 cells, two per word

Table 1. The term word. One 64-bit value: bit 63 flags a counted redirect, bits 62–56 hold the tag, bits 55–40 a 16-bit aux field (constructor id, function id, or block class), and the low 40 bits a heap location in words. Machine words, small naturals (up to $2^{48} - 1$), and packed one-field constructors own no node and are never counted or collected. HOLE is a reserved word marking an undelivered task slot, never a value.

```
case FID_SUM: {
  Term t = r0;
  if (term_tag(t) == PAK) {
    // one-word ctors are unboxed: the payload
    // lives in the term itself
    res = term_loc(t); goto exit;
  }
  // Node{l, r}: read both fields; the node
  // is freed, or handed to the arm as a spare
  Term l, r; spare = ctr_take(t, &l, &r);
  if (seq) {
    // two steps chained by stack frames,
    // the last one jumping into sum$j0
    push_frame(r, FID_SUM_STEP); r0 = l;
    goto again;
  } else {
    // a join task: one slot per child, each
    // child spawned pointing back at its slot
    Term j = task_new(FID_SUM_J0, 2);
    kid(j, 0, FID_SUM, l);
    kid(j, 1, FID_SUM, r);
    reply = j; // dealt by the scheduler
  }
}
```

The match takes ownership: `ctr_take` reads both fields and reclaims the node, because this reference was the only one, and the emptied node is offered as a *spare* to the arm’s own builds of the same size class. The two worlds are the same fork read twice: sequentially, the members run one after the other through stack frames and the last step jumps into the joiner; in parallel, the join task is born with one empty slot per child and the children are spawned pointing back at the slots their results will fill (Section 7). A *Leaf* never touches memory at all: a constructor with exactly one live word-sized field is packed into the term word itself.

header	monks	rings	stacks
--------	-------	-------	--------

corpus ... heap ...

lane words: free heads c0..c8 quantum counters
 heap: 128-word pages off one bump cursor ↑

Figure 1. The corpus: one flat word array shared by every CPU thread and the GPU. A 96-word header, per-lane scratch (allocator state, counters), the task rings, the device value stacks, then the heap, consumed by one global page cursor. Per-lane words are strided so device accesses coalesce.

4 Terms and Memory

4.1 The Term Word

A term is one 64-bit word (Table 1). Machine words, 32-bit floats as raw IEEE-754 bits, and naturals up to $2^{48} - 1$ are unboxed; a nullary or single-word-field constructor packs its payload into the location bits and owns no node. Every other term points at its *node*, its children in consecutive heap slots. A closure stores its captured arguments; a zero-capture closure is a bare function id with no node. A task node is the fork’s join, carried as an ordinary term (Section 7). An array block stores one term per word and owns its cells; a buffer is the same block shape for packed 32-bit words, two cells per word, so copying or dropping it touches no cell. The tag’s top bit is the sharing flag: a shared term keeps its tag and aux and points, through one redirect word, at the node it shares (Section 5).

4.2 The Corpus

All executors share one flat 64-bit word array, the *corpus* (Figure 1): a 96-word header, a scratch region per lane (the *monk*: its allocator words and counters, persisted across GPU dispatches), the task rings of Section 7, the device machine stacks, and the heap. A term crosses the CPU/GPU seam unchanged, in $O(1)$: no serialization, no copy, no pointer rewriting, and heap locations are stable for the whole run. The heap is carved into 128-word pages taken off a single global bump cursor, checked against a limit: the wired edge on the device, the memory cap on the host. There are 2^{14} lanes, a 128 by 128 grid, on every target; `-parallel` off runs the same machinery at width one, a single thread serving the grid, with no GPU.

The allocator keeps, per lane and per size class, two words: a LIFO free-list head and a partially consumed quantum of the lane’s last claimed page. There are nine small classes (2^0 to 2^8 words) and 23 huge classes (whole page spans). An allocation pops its class, else bumps its quantum, else claims a fresh page off the global cursor; a free is two plain stores onto the class list. Frees are exact, and every span goes back at the class it was taken from: a match frees the node it consumed or hands it to the arm as a spare, and an array or buffer frees its block whole. Huge spans recycle through one lock-free page stack per class, a pop briefly locking the head while pushes wait it out. Because the per-lane state rides in the monk scratch, whichever executor next serves a lane adopts its piles wholesale, and

freed memory recycles with no cross-thread free list on the allocation path.

Exhaustion is fail-stop by construction. A claim past the limit posts a numbered error into the header, once, and yields the *doomed page*: page zero, pre-filled with self-pointing constructors, so device threads chasing pointers after an out-of-memory stay inside it until the error poll drains the kernel; no thread waits, parks mid-allocation, or grows memory itself. On the device the story continues in Section 9.2: the host grows the wired window between dispatches and restarts the run whole, which a pure program cannot observe. Near the wired edge a lane instead *parks*: it hands its runnable work back to its ring and ends its round, so the host can wire a larger window before the next one.

5 Ownership at Run Time

Generated code manages memory with three operations. *Take* consumes a value at a match: load the fields and free the node, handing the freed slots back so a branch that builds a node of the same size class reuses them in place. *Keep* gives a value an extra owner: the first keep mints a *redirect*, a one-word cell holding the target address and a reference count, rewrites the local to the redirected form, and hands out a counted alias; the count lives in the redirect, never on the node itself, so an unshared value carries no count anywhere. *Drop* releases one ownership: a word owns nothing and costs nothing; a counted pointer decrements, and only the reference that takes the count to zero goes on to collect; a sole owner collects at once.

Take respects sharing. On a sole owner it reads the fields and reclaims the node; at count one it collapses the redirect and owns the node; above that it copies the fields out, sharing each one, and fades this reference, the last reference out collects. Every decrement is a release, and whoever sees zero acquires first, so field reads never race a free. Array and buffer blocks copy on write instead: a write to a shared block copies the block first, sharing each cell.

Which types pay any of this is decided at compile time, by two global analyses. *Share inference* decides which constructor types wear counts at all, to a fixpoint over every binder site: a type is hot when some binder of it is used more than once non-trivially or is reachable through a hot type’s live fields, and only hot constructors seal their stored words and are destructured through the counted take; everything else is built and consumed through plain stores. *Borrow inference* decides which arguments are only read: each such argument is lent raw, the callee’s takes become bare field loads, a field read off a borrowed value is itself borrowed, transitively, and the owner frees once, after the last reader, which is sound because work is never stolen and a join outlives its children. A fold, a lookup, a checksum walk over a reused tree costs no count traffic at all.

Dropping is one compiled walk, the runtime’s single collector: an iterative traversal that threads its worklist through the nodes being freed, each node’s first word displaced by the parent link, so the walk itself allocates nothing and

runs synchronously from any code; shared terms decrement and stop unless they were the last owner. There is no deep cloner and no copy intrinsic anywhere in the runtime: a `+` variable is emitted at every use because its type is `Data` by declaration, and each extra use is the borrow or the share the compiler placed. There is no tracing, no epochs, and no deferred queues: the whole discipline is plain malloc-free-shaped code, the frees placed by the types, plus a compiler-placed count exactly where a second owner genuinely escapes.

6 Arrays

In the source, `Array<T>` is an ordinary datatype, a perfect binary tree walked by index bits, built by `new(d, v)` with 2^d leaves and accessed through `get`, `set` and a generic swap that returns the displaced element. The backends intercept it: an array of machine words runs as a *buffer*, one flat block of packed 32-bit cells, two per word, and an array of anything else as a *block* of one term per word, each cell owned by the block. Creation allocates the block whole; a read is an indexed load, a write an indexed store, a swap both. Affinity is what makes the in-place access sound: an unshared array has one owner, so nobody can observe the mutation, and a shared one is copied on write, cell by cell, before the store. Block depth is capped at 31 on every backend, one wall, identically; past it creation refuses on all three executors.

7 The Task System

7.1 Tasks are Terms, in the Heap

A task is not a runtime object beside the program; it *is* a heap term. A fork compiles to a *join task*: one heap span of arity plus two words, `[args..., cont, idx|rem]`. The argument slots of children that have not answered yet hold the reserved `HOLE` word; `cont` holds the parent continuation, itself a task term, or `HOLE` at the root; and the last word packs `idx`, which slot of the parent this task's own answer fills, beside `rem`, how many children still owe one. The arity comes from the function id, so the node carries no length. A delivery writes its slot and decrements `rem` with release ordering; the thread that takes it to zero owns the now-runnable task and runs it. All children of a join may deliver concurrently; they write disjoint slots and race only on the countdown. When a whole cascade runs inside one lane, the countdown runs in plain memory. A `HOLE` continuation is the root: its delivery is the program's answer.

7.2 The Ring Cube

The task frontier lives in the *cube*: a hardcoded 128 by 128 square of rings, one ring per lane, 2^{14} on every target, served by one thread when parallelism is switched off, the same code at width one. Each ring is a fixed FIFO with put and get counters, and each owns its lane's state: the allocator words, the machine-stack window, the counters. A slot hands a task across threads with one publication protocol: the producer fetch-adds the put counter, writes the slot's low half relaxed,

ring 0	t0	t4	t8	
ring 1	t1	t5	t9	
ring 2	t2	t6	w	
ring 3	t3	t7		

Figure 2. The cube, shown 4 lanes wide. Tasks pushed along a row during one phase are drained down the columns of the next: the flip swaps the index map, an $O(1)$ transposition that moves no task. A task born during a work pass (*w*, darker) lands past the snapshot and belongs to the next wave.

then release-stores the high half carrying the lap parity in its top bit; a consumer's acquire load rejects a wrong lap as unpublished. The square is stored slot-major, so a row and a column are both $O(1)$ index maps, and *flipping* the cube, turning the rows one phase filled into the columns the next phase drains, is swapping the indexer, not moving data (Figure 2). A task is written into a ring once, when it is born or when a continuation returns to the cube; no phase re-reads a task to place it somewhere else: no redeal, no migration, no rebalancing pass. Global pushes advance one shared cursor whose total is the frontier estimate the driver reads.

7.3 Seed, Grow, Work

Evaluation alternates three phases over the cube until the root answer lands, selected by the frontier estimate f : seed below 128, grow below 2^{14} , work at saturation.

Seed runs the small frontier on one group of 128 lanes: each pops a task, runs it one fork step, and pushes the fork's children back into the strip. Forks double the row's population, and seeding ends when every ring of the row is non-empty. On the host there is no seed twin: the solo prologue covers the small frontier, and for small programs it finishes the evaluation by itself, without ever dispatching.

Grow runs below saturation: 128 groups run every row exactly as seed runs the top one, one thread, one ring, pop, step, push, through the flip, so the tasks one phase dealt along its rows drain down the next phase's columns. Growth skips the definitions that cannot fork, they can never widen the frontier and wait for work, and the lanes vote: growth stops when every ring has work or nothing grew. The host mirrors the same rule on its worker pool.

Work runs at saturation: every lane drains exactly the frontier its ring held at the turn's start, sequentially, forks running as ordinary consecutive calls. New tasks born during the turn land past the snapshot and belong to the next wave: work consumes exactly the frontier that existed when it began. When a lane's machine cannot finish, its answer must wait on other lanes' deliveries, its continuation returns to the cube as a task, claimed by one global increment, which fills the first rows; the flip then spreads those rows into columns for the next turn. A lane that is growing dangerously close to the wired edge, or that is parked by the memory protocol of Section 4.2, pushes its runnable work back to its ring and stops, and memory is grown between phases, never inside one. This is the bulk-

synchronous rhythm [17]: bursts of exponential fork, then deep sequential focus, then the next wave. The driver reads and clears the push cursor each round and stops when the root has delivered; an empty frontier with no root answer is itself a numbered error, never a hang.

8 Sequential Execution

The work phase runs whole calls, so most of a program's time is spent in ordinary sequential code. Two constraints shape it: GPU device code has no recursion, so native C recursion is unavailable there; and one text must serve both worlds, so every call site carries both readings.

All compiled segments live in *one* worklist function. On the host, segments are labels and dispatch is a computed goto; on the device, the same segments sit in a switch inside a poll loop, so a posted error drains every lane. The machine's state is an argument register bank, a result register, the current function id, and a stack; there are no C call frames for user code anywhere. A segment's prologue loads its parameters from the bank, frees the task node it entered from, and runs its case tree; it ends by answering: a value delivered through its continuation, a tail jump into another segment with the bank loaded, a cut, or a fork.

The *dual call protocol* is the one mechanism behind Section 3's claim that one text serves three executors. At a cut, the sequential world pushes a frame holding the continuation's captures and function id, jumps into the callee, and on return pops the frame and enters the continuation with the result in the result register; the parallel world instead allocates a continuation task expecting one result and replies it to the scheduler; and a marked call returns a spawned task rather than jumping, spawn, do not nest. At a fork, the parallel world births the join task of Section 7; the sequential world runs the same members as consecutive steps chained by stack frames, the last step jumping into the joiner. Minted continuations are *sequential* segments: their arguments travel on a value stack rather than the register bank, and when the machine pops one it runs it in place, its frame pushed and control jumping straight in, with the result already in the result register. Self tail calls reload their parameters and spin inside their own segment; spare nodes flush before any jump or return.

The machine is kept fast by four compiler disciplines. Parameters of word and float type are *unboxed*: they live as raw machine words in the bank, rewrapped at uses, so arithmetic chains never touch the heap. Two match specials remove branches: a two-arm boolean match whose arms are constant equality tests becomes a single branchless test over an OR-mask, and a closed chain of constant natural cases becomes a deduped constant table with a clamped lookup. *Fusion* inlines a non-recursive callee's text at a tail call, and inlines a pure cut whose callee returns a word or one flat constructor, spinning a recursive one as a loop in place, its result consumed as unpacked parts so the record never allocates; on the device the spun loop is outlined into its own function, since device compilers degrade superlinearly

in single-function size. And in main's chain a small call-free callee is inlined even across a constructor match, because one extra segment in the worklist function costs about two percent of whole-function code generation, more than the inlined text.

Recursion depth is bounded by the stack: host workers run on guarded mmap-ed stacks, and a fault is trapped through an alternate-stack handler and reported as the same numbered depth error the device posts when its value stack overflows. The same program at the same depth answers the same way on every executor.

9 The GPU

9.1 One Source, Its Own Shader

The runtime speaks to one GPU API, Metal, and the device code is not a port: at launch, the binary compiles *its own C source file* as the Metal shader library, so host and device run the same functions by construction, and a bug fixed once is fixed everywhere. One macro family covers the concurrency layer: on the host it expands to the compiler's real atomic orders, on the device to relaxed 32-bit atomics bracketed by sequentially consistent fences, always on the low 32-bit half of a corpus word, whose value fits 32 bits for every shared protocol (the ring counters, the join countdown, the error word). Full 64-bit terms cross threads as two halves ordered by the publication lap bit and the join's fill countdown (Section 7).

A device cannot abort, so failure is a protocol: the first failing lane compare-and-swaps its numbered error into a header word, once; every unbounded device loop polls that word and drains; and the host reads it after the dispatch, prints one line, and exits. Genuine memory exhaustion inside a kernel follows Section 4.2: the doomed page keeps every wandering lane harmlessly inside one page until the poll fires, so a dispatch that cannot progress fail-stops instead of hanging.

9.2 Memory Discipline

The corpus is one shape on both processors: the same host mmap wrapped, zero-copy, in a Metal buffer at the same addresses. Address space is nearly free, but every byte a kernel may touch must be *wired* first, a real per-gigabyte cost, so the window is wired in *tomes* of 256 MB: at least two from the start, and before every round the host wires ahead of use, three tomes past the allocation cursor and at least doubling, never during a dispatch. The arena itself is sized from the device's own limits at reserve time.

When a kernel still runs out, the wait-and-resume protocol of an earlier design is replaced by something simpler: the error drains the dispatch, and the host *re-feeds and restarts*, rewiring a larger window, resetting the scheduling regions, reseeding the heap, and rerunning the program's root from scratch. A pure program cannot observe the retry, and the retry is legal only before any effect has run (Section 10); when the device is truly out of memory, the restart fails

loud with the same numbered error. This trades a bounded amount of recomputation for the absence of an entire in-flight parking protocol, and it is the reason no device thread ever waits on memory.

9.3 Whole Phases as Dispatches

The GPU driver never round-trips per step. A seed is one dispatch of one threadgroup; a grow is one dispatch of 128 groups; a work pass is one dispatch of one thread per lane; between dispatches the host’s whole job is to read the phase counters, wire ahead, and pick the next phase. Results never copy, since host and device share the corpus, and the per-lane state that must survive a dispatch boundary, allocator words, counters, is exactly what the monk scratch persists. The CPU and GPU evaluators interoperate over the one heap, but never concurrently: exactly one evaluator owns the heap at any moment, by construction.

9.4 Numeric Parity

One semantics everywhere extends to floats. Device compilers default to fast math: relaxed f32 algebra, approximate roots, cross-statement fused multiply-adds, each of which alone breaks bit parity with the CPU build. The device compilation at launch therefore pins safe math and contraction off and uses the precise square root, truncation to an integer goes through the bit pattern, and the compiler emits f32 arithmetic one operation per statement so the host compiler cannot contract either. Host and device agree bit for bit on every primitive; shifts past 31 answer zero, division by zero follows the base library’s own laws (the quotient is zero, the remainder the dividend), and the same laws hold in the JavaScript backend, so all three executors print byte-identical output.

9.5 Honoring the Placement Mark

On a host with a usable device, the event loop honors the mark at its own level: a marked call met at a sequential program point, nothing else running, one root, is detached whole to the Metal evaluator, which seeds the root into the cube, runs the phase loop on the device, and delivers the answer on the shared heap. The mark therefore composes: the marked call’s subtree forks internally on the device along its own fork lets, the surrounding program continues on the CPU, and a mark reached mid-computation degrades to a spawn or a plain call rather than serializing the machine (Section 2.3).

10 The Event Loop

Every compiled program is an event loop anchored on the user’s monadic IO main; the runtime accepts no other main type and has no way to stringify arbitrary terms, a program prints what it chooses to print, through IO. The loop runs on one CPU thread and orchestrates the pure evaluators: at each bind it hands one saturated call to the chosen evaluator, blocks for the pure answer, runs the effect the program requested, print, files, TCP and UDP sockets, environment,

bench	seq CPU	par CPU	Metal GPU
tree bitonic sort	5.14 s	0.93 s	1.37 s
game of life	4.21 s	0.70 s	0.11 s
k-means	5.00 s	0.93 s	0.64 s
mandelbrot	4.93 s	0.74 s	0.14 s
tree matmul	2.05 s	0.37 s	1.00 s
merkle tree	4.65 s	0.79 s	0.35 s
n-body	4.88 s	0.75 s	0.09 s
n-queens	3.93 s	0.60 s	2.01 s
tree radix sort	3.77 s	0.69 s	0.97 s
raytrace	4.96 s	0.85 s	0.45 s
symbolic regr.	2.79 s	0.49 s	0.80 s
terrain	3.59 s	0.55 s	0.35 s

Table 2. The pinned suite. Seconds on one Apple M4 Mac Mini: one thread, 16 threads, and the integrated GPU through Metal. Every cell pays a warm run plus a timed run, by law; the committed result is a pin diffed against thereafter. The matmul GPU cell is marked unstable in the pin file: its variance is real and under investigation.

each a foreign import compiled beside the program, and feeds the continuation. Effects never run inside an evaluator, and the machine performs no IO: this is what makes the whole-run memory retry of Section 9.2 legal, and it is enforced, not assumed. The loop is light, no per-call setup, so effect-dense programs, a web server, stream through it; the same effect kit backs the C runtime and the JavaScript backend, one file per effect, and the CLI is four knobs: `-threads`, `-parallel`, `-gpu`, `-help`.

11 Evaluation

The suite is twelve benchmarks, each a self-contained Bend program whose workload and expected checksum are fixed in its header, so every number here is reconstructible from the shipped sources, which live under `bench/runtime/` in the Bend repository. They span tree-shaped sorting (bitonic, radix), dense numeric grids (game of life, mandelbrot, n-body, terrain), clustering and regression search (k-means, symbolic regression), Merkle hashing, quad-tree matrix multiplication with Freivalds’ verification, backtracking search (n-queens), and recursive raytracing.

Method: every cell runs a warm run plus a timed run on an otherwise idle machine, and every executor of a bench must reproduce the header’s checksum, which is how cross-target semantic drift (a fast-math flag, say) has repeatedly been caught. Benchmarks are never hand-shaped toward a backend (a committed suite law): each is the idiomatic spelling of its algorithm, and a backend that punishes that spelling keeps the loss; the same law bans any optimization that fires only on the benchmark corpus. The parallel column is 16 threads (the runtime’s default is the core count). The

committed result is a *pin* diffed against thereafter, so slow regressions cannot hide inside run-to-run noise.

Table 2 supports three readings. First, the parallel CPU column is a 5.4x to 6.7x speedup across the suite, which is what the no-balancing design delivers when the contract holds: the suite forks equal halves, and the wave ends together. Second, the GPU column is bimodal. Uniform-work benchmarks reach large multiples of the sequential build on the integrated GPU, n-body 54x, game of life 38x, mandelbrot 35x, merkle 13x, and raytrace joins them at 11x; divergent and skewed workloads lose to the parallel CPU build, n-queens most visibly, with bitonic, matmul, radix and symbolic regression also behind it, and the design stance is to report them as losses rather than tune the scheduler toward them. Third, all parallel numbers are self-relative: no external parallel implementation is raced here, and the sequential column is the anchor the speedups multiply.

12 Related Work

Interaction nets. The author’s earlier runtimes, HVM and its successors, evaluate interaction combinators [7, 8, 13]: graph rewriting with strong confluence, optimal sharing, and parallelism at every redex. Bend’s runtime contains no interaction nets. The lineage survives in the goals (massive implicit parallelism, one code on CPU and GPU) and in hard lessons about GPU memory discipline, but the mechanism is replaced wholesale: affinity from the type system gives unique ownership directly, so the graph, the sharing nodes, and the duplication machinery all become unnecessary. What was lost is optimal reduction of shared redexes; what was gained is native-speed sequential code, flat memory, and a cost model a programmer can read.

Task scheduling. Work stealing [2, 4] is the standard answer to irregular parallelism, and its deliberate absence here is the main scheduling novelty: Bend moves the balance obligation into the language contract (Section 2.2) and keeps the runtime wave-structured [17], which is what makes one scheduler design run on a GPU at all. Lazy task creation [11] anticipates the work phase’s discipline: a fork met while draining runs as ordinary sequential calls, excess parallelism represented, not spawned. GHC’s sparks [9] and Multilisp’s futures [5] are hint-driven like the fork *let*, but both back onto stealing pools and a garbage-collected heap.

Functional GPU compilation. Futhark [6], Accelerate [3], and the NESL line [1] compile array-level data parallelism to GPU kernels, with flattening as the central transformation. Bend differs in scope and shape: the unit of GPU execution is not a kernel compiled from an array combinator but the whole language, recursion, allocation, and algebraic data types included, run by a scheduler that lives on the device; and the same binary serves CPU and GPU from the same intermediate form. The price is that Bend’s GPU code is younger and less specialized than a flattening compiler’s output, visible in Table 2’s losses.

Memory management. Region inference [15] and linear types [18] both aimed at collector-free functional mem-

ory; Rust [10] made ownership mainstream with borrows checked statically. The reference-counted functional runtimes, Lean’s [16] and Koka’s Perceus [12], are the closest in mechanism to Bend’s shares: precise counts, reuse, and no tracing. Bend differs in where the counts live and when they exist at all: its type system makes affinity the default and admits contraction only at the reusable kind *Data*, so counts are minted only where a whole-program analysis finds genuine sharing, ride in a separate redirect word rather than on the object, and the unshared majority of the program compiles to moves, borrows, and frees with no count traffic whatsoever.

13 Limitations

The equal-halves contract is undecidable and unverified: a skewed program silently loses its parallelism, lanes idle at the end of a work turn, and the runtime deliberately declines to repair it. The GPU story is uneven: divergent workloads lose to the CPU (Table 2), the matmul GPU cell shows real unexplained variance, the CPU and the GPU never compute concurrently (a marked call is honored only at sequential program points), and the backend is Metal on a current macOS, one API, one platform, today; a Linux build and other device APIs are future work, not present tense. The whole-run memory retry is legal only before the first effect; an effectful program that exhausts the device fails loud instead. Reference counts saturate at a fixed width and saturation is a numbered fail-stop, not a fallback. f32 arithmetic is bit identical across executors but transcendentals beyond the pinned set are not covered. The runtime is young, and none of the C code is verified: the correctness argument is the checksum discipline plus the type system’s guarantees, and the companion paper’s theorems stop at the calculus, not at this implementation.

14 Conclusion

BendRT compiles a pure functional language to one C file and runs it unchanged from a single thread to an integrated GPU: one worklist machine gives every executor its unit of work, the three-phase scheduler forks and drains a fixed grid of rings without ever contending, and affinity places the frees, so no collector and no stealing pool couples the lanes. The benchmarks show both sides of that bargain: 5x to 7x CPU scaling and large GPU wins on uniform work when the fork contract holds, and honest losses on divergent and skewed workloads, which the runtime deliberately declines to repair. The companion paper [14] develops the type theory this machine relies on.

Acknowledgments

Thanks to the Higher Order Company team for discussions.

References

- [1] Guy E. Blelloch. 1996. Programming Parallel Algorithms. *Communications of the ACM* 39, 3 (1996), 85–97.
- [2] Robert D. Blumofe and Charles E. Leiserson. 1999. Scheduling Multithreaded Computations by Work Stealing. *Journal of the ACM* 46, 5 (1999), 720–748.
- [3] Manuel M. T. Chakravarty, Gabriele Keller, Sean Lee, Trevor L. McDonell, and Vinod Grover. 2011. Accelerating Haskell Array Codes with Multicore GPUs. In *Declarative Aspects of Multicore Programming (DAMP)*, 2011. ACM, 3–14.
- [4] Matteo Frigo, Charles E. Leiserson, and Keith H. Randall. 1998. The Implementation of the Cilk-5 Multithreaded Language. In *Programming Language Design and Implementation (PLDI)*, 1998. ACM, 212–223.
- [5] Robert H. Halstead. 1985. Multilisp: A Language for Concurrent Symbolic Computation. *ACM Transactions on Programming Languages and Systems* 7, 4 (1985), 501–538.
- [6] Troels Henriksen, Niels G. W. Serup, Martin Elsman, Fritz Henglein, and Cosmin E. Oancea. 2017. Futhark: Purely Functional GPU-Programming with Nested Parallelism and In-Place Array Updates. In *Programming Language Design and Implementation (PLDI)*, 2017. ACM, 556–571.
- [7] Yves Lafont. 1990. Interaction Nets. In *Principles of Programming Languages (POPL)*, 1990. ACM, 95–108.
- [8] Yves Lafont. 1997. Interaction Combinators. *Information and Computation* 137, 1 (1997), 69–101.
- [9] Simon Marlow, Simon Peyton Jones, and Satnam Singh. 2009. Runtime Support for Multicore Haskell. In *International Conference on Functional Programming (ICFP)*, 2009. ACM, 65–78.
- [10] Nicholas D. Matsakis and Felix S. Klock. 2014. The Rust Language. In *High Integrity Language Technology (HILT)*, 2014. ACM, 103–104.
- [11] Eric Mohr, David A. Kranz, and Robert H. Halstead. 1990. Lazy Task Creation: A Technique for Increasing the Granularity of Parallel Programs. In *LISP and Functional Programming (LFP)*, 1990. ACM, 185–197.
- [12] Alex Reinking, Ningning Xie, Leonardo de Moura, and Daan Leijen. 2021. Perceus: Garbage Free Reference Counting with Reuse. In *Programming Language Design and Implementation (PLDI)*, 2021. ACM, 96–111.
- [13] Victor Taelin. 2024. *HVM2: A Parallel Evaluator for Interaction Combinators*. Retrieved from <https://github.com/HigherOrderCO/HVM2/blob/main/paper/HVM2.pdf>
- [14] Victor Taelin. 2026. *BendTT: An Affine Dependent Type Theory*.
- [15] Mads Tofte and Jean-Pierre Talpin. 1997. Region-Based Memory Management. *Information and Computation* 132, 2 (1997), 109–176.
- [16] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *Implementation and Application of Functional Languages (IFL)*, 2019. ACM.
- [17] Leslie G. Valiant. 1990. A Bridging Model for Parallel Computation. *Communications of the ACM* 33, 8 (1990), 103–111.
- [18] Philip Wadler. 1990. Linear Types Can Change the World. In *Programming Concepts and Methods*, 1990. North-Holland, 561–581.